## Github Repo

https://github.com/su92-bsaim-f23-101-collab/Assembly-Project-Sorting-vis

**Group Member:**
Ali Afzal – BSAI-6B-102

Muhammad Hassan -BSAI-6B-101

## Sorting Benchmark Tool

This project implements a **sorting algorithm visualizer** using **NASM** and the **DOSBox** environment. The program demonstrates six classical sorting algorithms through a real-time animated graphical interface in text mode.

The visualizer displays sorting behavior using vertical bars, where each bar represents a numeric value. During execution, the bars change color to indicate comparisons, swaps, and completion states. This provides a practical understanding of algorithm performance and time complexity.

The implemented algorithms are:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Shell Sort
- Quick Sort
- Merge Sort

The system also measures execution statistics such as:

- Total comparisons
- Total swaps
- Execution time

---

**Table of Contents**



---

## 1. Introduction

Sorting is one of the most fundamental operations in computer science. Understanding sorting algorithms becomes easier when their execution can be visualized.

This project presents an educational visualizer in **8086 assembly language** that shows sorting procedures in an animated manner. The project uses BIOS interrupts for:

- keyboard handling
- screen drawing
- timing
- colored output

The system is designed as a .COM executable for DOS.

---

**2. Objectives**

The objectives are:

- Learn low-level programming
- Understand memory handling in assembly
- Implement multiple sorting algorithms
- Compare performance
- Visualize operations
- Use BIOS interrupts
- Build interactive DOS application

---

**3. Problem Statement**

Sorting algorithms are often difficult to understand from theoretical explanation only. Students require a practical and visual representation.

This project solves that problem by:

- generating random arrays

- displaying bars
- animating swaps
- comparing performance metrics

---

**4. System Requirements**

**Hardware**

- PC/Laptop
- 2GB RAM minimum
- Keyboard

**Software**

- NASM
- DOSBox
- Text editor
- Windows/Linux

---

**5. Development Tools**

**NASM**

Used for assembling source code.

Compile command:

**DOSBox**

Runs the 16-bit .COM program.

---

**6. Program Architecture**

The program is divided into modules.

**Main Modules**

**User Interface Module**

Handles:

- menu
- title
- help screen
- results

**Sorting Engine**

Handles:

- algorithm execution
- statistics
- timing

**Visualization Engine**

Handles:

- bars
- animation
- color transitions

**Utility Module**

Handles:

- random numbers
- printing
- delay

- screen clearing

---

## 7. Algorithm Implementation

---

### Bubble Sort

$$f(n) = n^2$$

Compares adjacent elements.

**Working**

- Compare neighbors
- Swap if needed
- Repeat until sorted

**Complexity**

- Best: O(n)
- Average: O(n²)
- Worst: O(n²)

---

### Selection Sort

Searches minimum and swaps.

**Complexity**

- O(n²)

---

### Insertion Sort

Inserts each element in sorted position.

**Complexity**

- $O(n^2)$

---

**Shell Sort**

Improved insertion sort using gaps.

**Complexity**

- O(n log n)

---

**Quick Sort**

Uses divide and conquer.

**Complexity**

- Average: O(n log n)
- Worst: $O(n^2)$

---

**Merge Sort**

Uses splitting and merging.

**Complexity**

- O(n log n)

---

**8. Key Features**

**Real-Time Animation**

Bars redraw continuously.

**Color Indication**

- Red = swap
- Green = sorted
- White = normal

**Random Dataset**

60 random values generated.

**Statistics**

Measures:

- comparisons
- swaps
- execution time

---

**9. Code Explanation**

---

**Entry Point**

Defines COM executable format.

---

**Menu Section**

Displays algorithm options.

Implemented in:

Functions:

- selection
- help
- run all

- exit

---

**Drawing Engine**

Implemented in:

Purpose:

- draws all bars
- updates colors
- prevents flicker

---

**Random Generation**

Implemented in:

Uses linear congruential generator.

Formula:

$$X_{n+1} = (aX_n + c)\, mod\, m$$

---

**10. Data Structures**

Main arrays:

**data**

Stores actual values.

**tmp**

Used by merge sort.

---

**11. Procedures**

Major procedures:

| Procedure | Purpose |
|---|---|
| show_menu | menu |
| visualize_sort | start |
| draw_bars | graphics |
| fill_random_array | data |
| bubble_sort_animated | bubble |
| selection_sort_animated | selection |
| insertion_sort_animated | insertion |
| shell_sort_animated | shell |
| quick_sort_animated | quick |
| merge_sort_animated | merge |

---

**12. Testing**

Tested using:

- random input
- repeated execution
- run all mode
- help mode
- invalid keys

---

**13. Results**

Program successfully:

✓ compiles
✓ runs
✓ animates
✓ sorts
✓ measures

Output includes:

- before bars
- during bars
- final bars
- statistics

---

**14. Advantages**

- educational
- visual
- efficient
- interactive
- low memory
- practical learning

---

**15. Limitations**

- text mode only
- 16-bit limitation
- no mouse

- fixed size array
- limited colors

---

**16. Future Improvements**

Can add:

- heap sort
- radix sort
- graph mode
- sound
- user input size
- custom arrays
- speed control

---

**17. Conclusion**

This project demonstrates how advanced algorithms can be implemented in low-level programming.

It combines:

- assembly programming
- BIOS interrupt usage
- visualization
- algorithm analysis

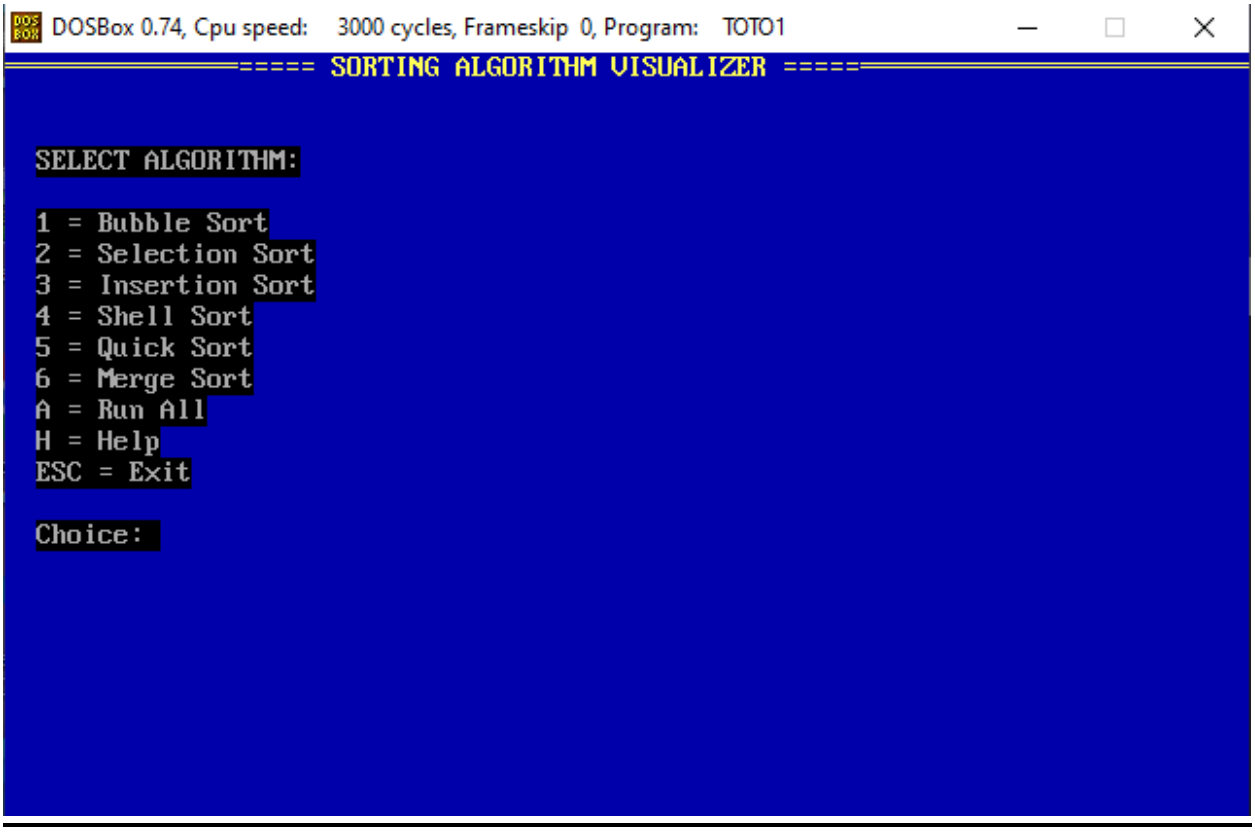
The project provides strong understanding of:

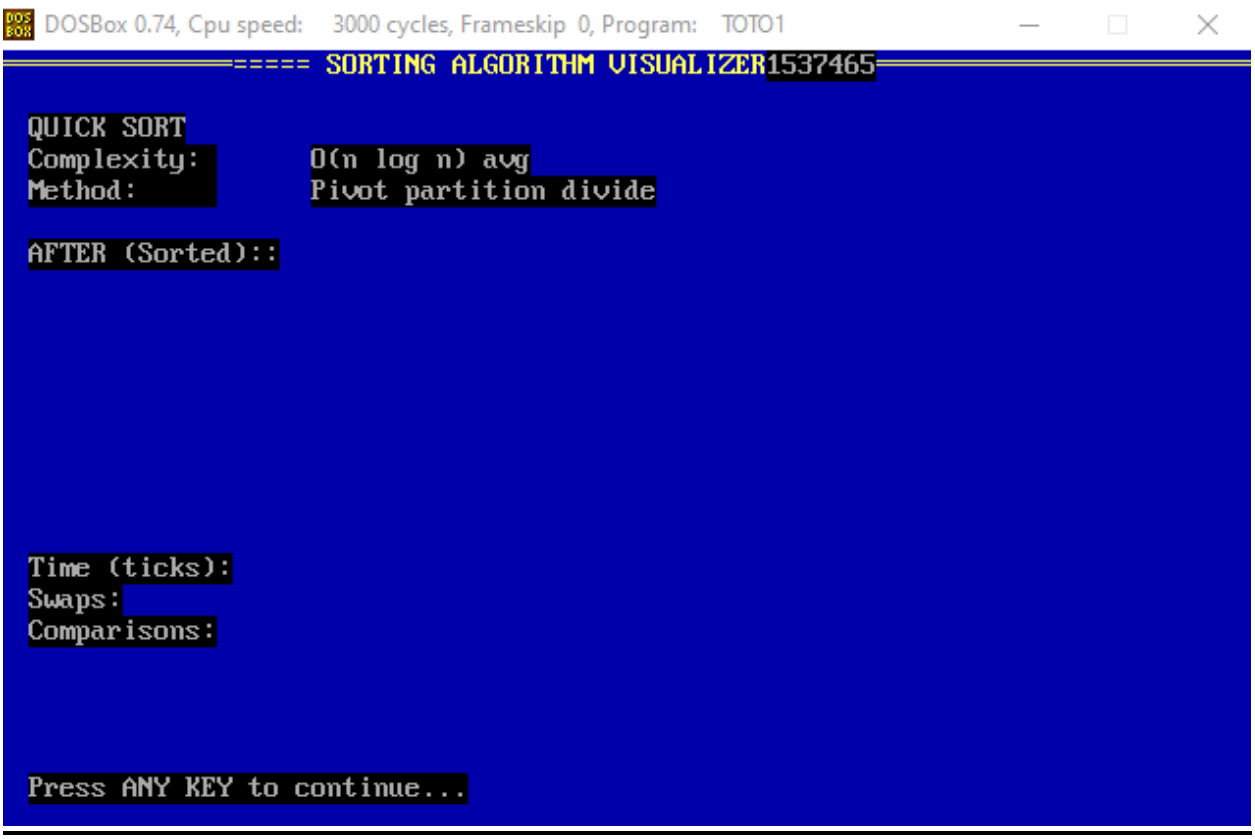
- sorting mechanics

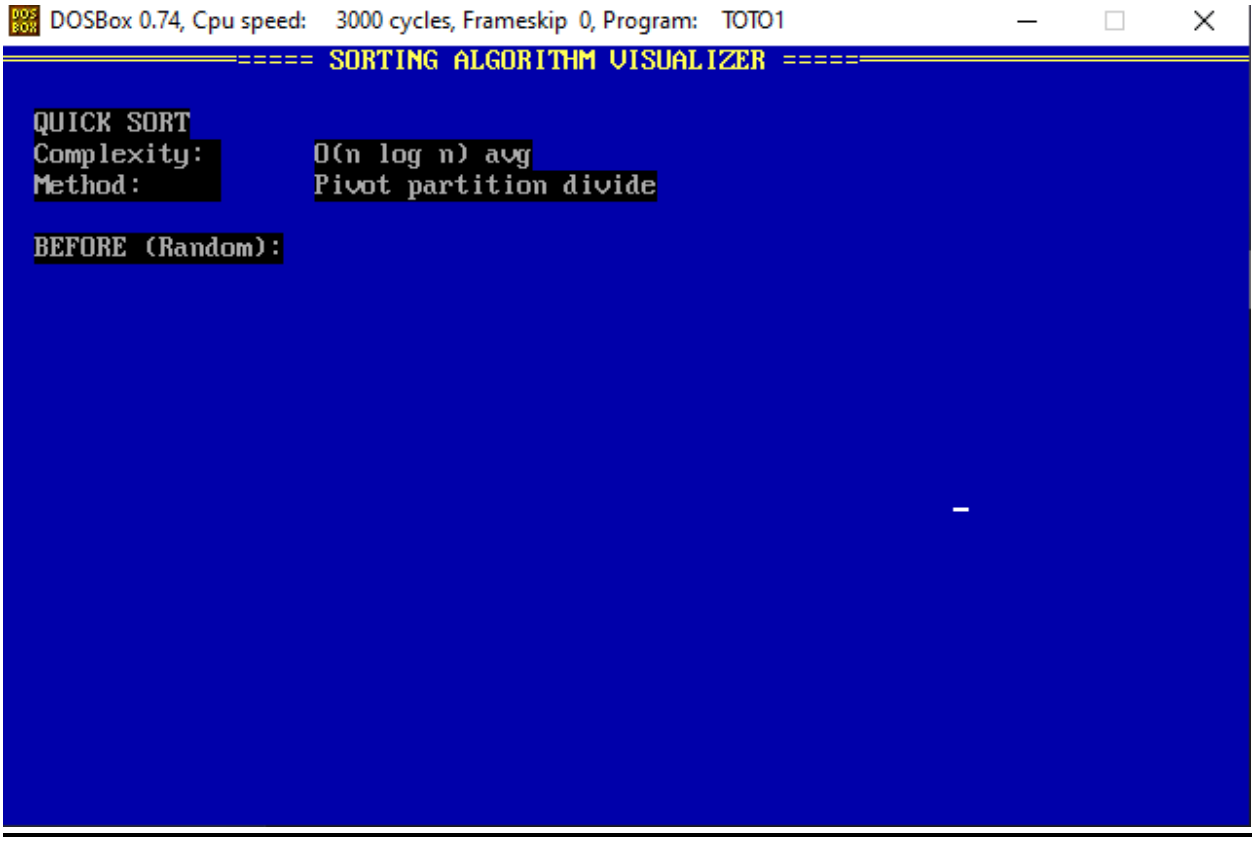
- memory operations
- CPU instructions
- procedural design

It proves that even 8086 assembly can create interactive and educational software.

## ScreenShots




DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: TOTO1
===== SORTING ALGORITHM VISUALIZER1537465
QUICK SORT
Complexity: O(n log n) avg
Method: Pivot partition divide
AFTER (Sorted)::
Time (ticks):
Swaps:
Comparisons:
Press ANY KEY to continue...


DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: TOTO1
===== SORTING ALGORITHM VISUALIZER =====
QUICK SORT
Complexity: O(n log n) avg
Method: Pivot partition divide
BEFORE (Random):